1 Problem 1: Data Analysis Report

Here, the target is to explore statistical relationship between the bank base interest rates (IR), the rate of unemployment (Unem) and the rate of mortgage repossession (Rep). All the annual data used are downloaded from the website of the UK government and the Bank of England, with their date ranging from 1975 to 2013. In this report, we first format and clean the raw data, followed by feature forward selection and model comparison. At last, model assumptions are tested and conclusions are drawn and explained.

To begin with, since the intersection of the date range of the three datasets is from 1975 to 2013, all data out of this range is removed. Then, to compute Rep, we divide the # of repossession by the # of mortgage and convert its unit to %. After doing this, we merge all relevant data into a single dataframe indexed by date, which contains IR, Unem and Rep from 1975 to 2013 as shown below. It should be noted that the unit of all columns is %.

year	ir	unem	rep
1975	11.0000	4.5	0.095942
1976	11.1137	5.4	0.093010
1977	8.8772	5.6	0.083841

(a)

year	ir	unem	rep
2011	0.5	8.1	0.327653
2012	0.5	8.0	0.300425
2013	0.5	7.6	0.258359

(b)

Figure 1: (a) first three entries and (b) last three entries of the merged dataframe

Secondly, data format, missing value and extreme value are checked. All data format is correct, and there is no missing value and no extreme value (i.e. no value lies outside three std away from the mean value).

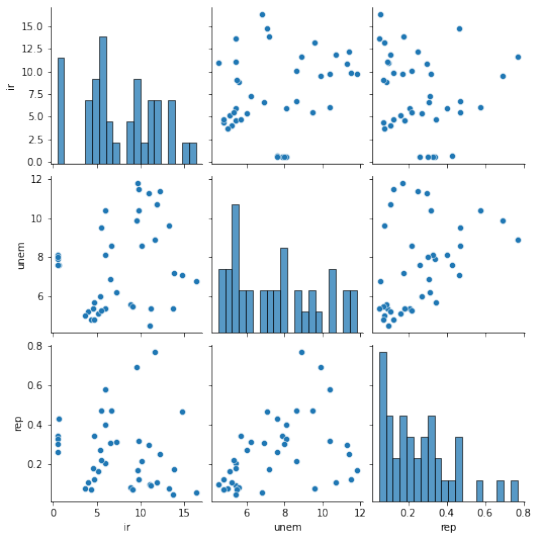


Figure 2: Distributions and dependencies of data

After cleaning the data, we take a look at the distributions and dependencies. From 2, there is no collinearity problem (and their correlation coefficients confirm this point), but the distribution of Rep looks like a log normal distribution. Therefore, a log transform is applied to Rep. We will use log(Rep) in later regressions.

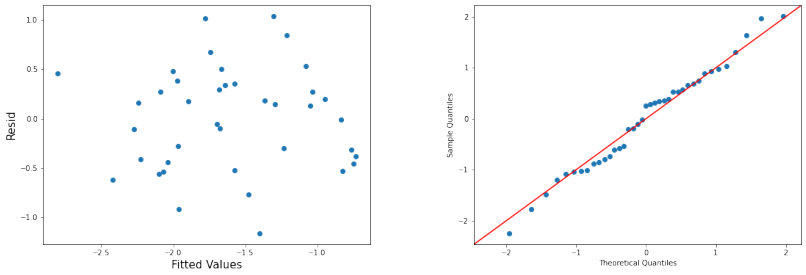
Now, all regression preparations have been done. To determine the dependent and independent variables in later regression analysis, we start from simple linear regression. More specifically, we regress each of the three variables on the rest two variables respectively. The results show that in each case the two independent variables' coefficients are both significant, but since when the dependent variable is log(Rep) the model has the highest R^2 (0.338) and lowest

AIC/BIC (75.80/80.79), we choose log(Rep) to be the dependent variable and the rest two to be independent variables in later regressions.

To add complexity to our former simple regression, we first add polynomial terms. Here a forward-style feature selection procedure is applied: each time we try adding polynomial term with one order higher than the present order to see whether all polynomial coefficients are significant. If they are, then repeat the process. If they are not, then this process is stopped. To avoid collinearity problem, orthogonal polynomials are used. Finally, the model with the best R^2 (0.498) and AIC/BIC (67.07/73.72) is log(Rep) $\sim$ 1 + IR + Unem + Unem^2, and as the procedure requires, all coefficients are significant (the largest P-Value is IR 0.011).

When it comes to spline regression, natural cubic splines are used. Again, the forward selection procedure is applied: each time we increase the degrees of freedom by 1 until a certain coefficient is insignificant. At last, no natural cubic spline regression is significant, i.e. even the lowest degrees of freedom leads to some coefficients being insignificant.

Now, from the perspective of coefficients significance, R^2 and AIC/BIC, the best regression model is the polynomial regression log(Rep) $\sim$ 1 + IR + Unem + Unem^2. Next we check its model assumptions. To see this, we plot the model residuals w.r.t. fitted values and residuals QQ-plot as follows:



(a) (b)

Figure 3: (a) residuals w.r.t. fitted values; (b) residuals QQ-plot

Also, considering that the P-value of the JB test is 0.782, we have enough confidence to conclude the polynomial regression model's residuals are normally distributed and have homoscedasticity. Therefore, model assumptions are met.

The coefficients of the above polynomial regression log(Rep) $\sim$ 1 + IR + Unem + Unem^2 are const -6.5791, IR -0.0578, Unem 1.2997, Unem^2 -0.0709 [1]. Based on this, following statistical conclusions can be drawn:

- There is negative quadratic relationship between Unem and log(Rep). More concretely, when Unem < 9.166, the higher the Unem, the higher the Rep. This is intuitive, since the higher the Unem, people will possibly have less money to pay their mortgage, thus leading to higher Rep. However, when Unem > 9.166, the relationship reverses, leading to a counter-intuitive statistical result. Therefore, more finer analysis in this region is needed in the future.

- There is also negative relationship between IR and Rep, i.e. statistically speaking, the higher the IR, the lower the Rep. This is again intuitive: in the UK most mortgage rates are fixed, so the higher the IR, the more people will be able to get from their investments (such as, bank deposit), leading to a lower Rep.

Finally, when doing the polynomial regression log(Rep) $\sim 1 + \text{IR} + \text{Unem} + \text{Unem}^2$ using normalized data (without intercept), the uncentered R^2 and P-value of the JB test do not change, being 0.498 and 0.782 respectively. So there is no improvement in these two aspects, as expected. But the AIC/BIC is worsened to 88.82/93.82. Therefore, from the perspective of AIC/BIC, normalization does not improve the proposed polynomial regression model. It even makes the regression worse.

[1] All the coefficients have been converted back to basis function$\{1, x, x^2\}$.

2 Problem 2: Python_DataQ2 Analysis

2.1 OLS Regression

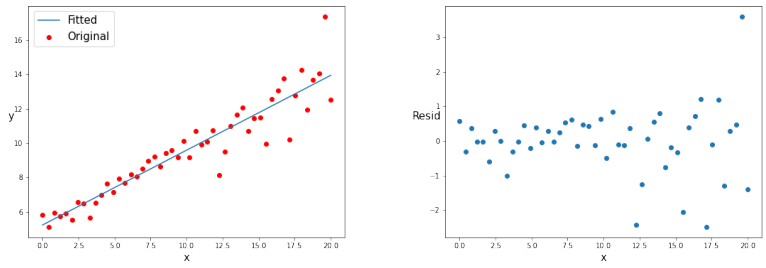


Figure 4: Fitted values and the original data (left) and residuals w.r.t. inputs x (right)

Remark. *According to the residuals, the linear regression model's homoscedasticity assumption seems violated, since the first half residual points seem have smaller variance than the second half residual points.*

2.2 WLS Regression

It should be noted that in the optimal WLS estimation, the weights used in optimization should be proportional to the inverse of error's variance. Therefore, the weights w that are really used should be computed as follows:

$$w = \frac{1}{weights^2}, \tag{1}$$

where *weights* are the column provided by the original dataset.

After WLS regression, results are presented below:

Param	WLS Coef.	Coef. Std Err.	OLS Coef.	Coef. Std Err.
const	5.2469	0.1426	5.2426	0.2707
x	0.4466	0.0180	0.4349	0.0233

Table 1: WLS and OLS regression coefficients

2.3 Comparison

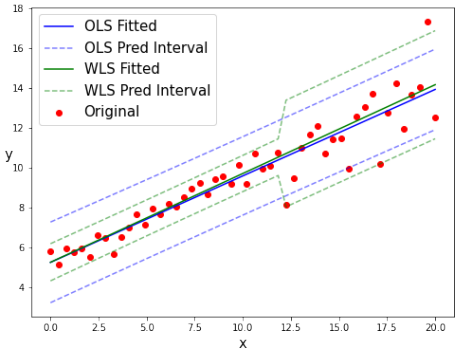


Figure 5: OLS vs WLS

Remark. *From figure 5, we see that the prediction interval of WLS grasps the change of error's variance.*

3 Problem 3: Optimal Spline Approximation

Here we assume cubic splines are used. And below polynomial regression is viewed as a special case of spline regression where no knots are involved, i.e. number of knots is 0.

The main idea of the program: we iteratively increase the number of knots by 1 and update the best (smallest) AIC and its corresponding number of knots (the maximal number of knots is set to be $\sqrt{length(x)}$). It should also be noted that only regressions whose coefficients are all significant will be recorded.

```
def OptimalSpline(y, x):
    '''
    Try to find the optimal spline approximation.
    '''
    def pielinear(x, knots, p=3):
        '''
        x: observations x_i
        knots: knots k_i
        p: max order of the polynomial (assume to be 3)
        '''

        res = np.vander(x, p + 1, increasing=True)
        nullArray = np.zeros([len(x), 1])
        x_T = x.reshape(len(x), 1)
        for k in knots:
            res = np.concatenate([res, (np.maximum(x_T - k, nullArray))**p],
                                 axis=1)
        return res

    x = x.copy()
    y = y.copy()
    best_num_knots = -np.infty
    best_AIC = np.infty
    for num_knots in range(int(np.ceil(np.sqrt(len(x)))) + 1):
        if num_knots == 0:
            x_reg = np.vander(x, 4, increasing=True)
        else:
            knots = [
                np.percentile(x, 100 / (num_knots + 1)) * i
                for i in range(1, num_knots + 1)
            ]
            x_reg = pielinear(x, knots)
        model = sm.OLS(y, x_reg).fit()
        # all coefficients should be significant
        if (model.pvalues <= 0.05).all() and model.aic < best_AIC:
            best_AIC = model.aic
            best_num_knots = num_knots
    return best_num_knots
```